Detyra Shtesë

Sqarim: Disa nga këto detyra lidhen direkt me temat që po shtjellojmë tani, disa lidhen me lëndët e tjera dhe disa vetëm me koncepte që kemi trajtuar më herët.

Kjo detyrë ka për qëllim modelimin e një fabrike që përdor robotë të llojeve të ndryshme për operacione të sigurisë, prodhimit, asistencës dhe administrimit. Fabrika ka një buxhet energjetik ditor dhe është e nevojshme të përzgjidhet një kombinim optimal robotësh që ofrojnë efikasitet sa më të lartë pa e tejkaluar këtë buxhet.

1. Klasa Abstrakte Bazë

1.1. Klasa abstrakte Robo

Kjo është klasa bazë për të gjithë robotët në fabrikë.

Atribute:

- String id *(read-only)*
- String tipi
- double energjia

Metodat:

- Konstruktor që inicializon atributet.
- Metoda get/set për atributet përkatëse (pa set për id).
- Metoda abstrakte:
    - double llogaritKonsum()
    - boolean eshteAktiv()
- toString() →
  <id> : <tipi> - <energjia> kWh

2. Llojet Kryesore të Robotëve

Të gjitha klasat e mëposhtme **trashëgojnë nga klasa bazë Robo**.

2.1. RoboSigurie – trashëgon Robo

Përdoret për patrulla sigurie.

Atribut:

- int numriPatrullave

Rregulla(metodat):

- Konsumi: energjia * numriPatrullave / 100
- Aktiv nëse numriPatrullave ≥ 1
- Efikasiteti: 3 * numriPatrullave

2.2. RoboProdhimi – trashëgon Robo

Përdoret për proceset e prodhimit.

Atribut:

- int kapaciteti *(njësi produktesh/ditë)*

Rregulla(metodat):

- Konsumi: kapaciteti * 0.30
- Aktiv nëse energjia ≥ llogaritKonsum()
- Efikasiteti: kapaciteti

2.3. RoboAsistence – trashëgon Robo

Përdoret për asistencë teknike ose humane.

Atribut:

- int niveliEmpatise (1–10)

Rregulla(metodat):

- Konsumi: 5 + (10 - niveliEmpatise)
- Aktiv nëse niveliEmpatise > 5
- Efikasiteti: 2 * niveliEmpatise

3. Hierarki e Zgjeruar e Robotëve Administrativë

3.1. Klasa abstrakte RoboAdministrativ – trashëgon Robo

Robotët administrativë nuk kryejnë funksione fizike si prodhimi apo patrullimi, por përdoren për menaxhim të të dhënave dhe proceseve administrative.

Atribut:

- int niveliInteligjences *(1–10)*

Metodat abstrakte:

- int llogaritPunenDitore() – sa njësi pune admin kryen roboti.

Konsumi i tyre llogaritet si:

- llogaritKonsum() = niveliInteligjences * 0.5 + 3

Robotët administrativë konsiderohen aktivë nëse:

- niveliInteligjences ≥ 4

3.2. RoboMenaxhimi – trashëgon RoboAdministrativ

Robot i avancuar për vendimmarrje.

Atribute shtesë:

- int nrVendimeve *(sa vendime mund të marrë në ditë)*

Rregulla:

- Puna ditore: nrVendimeve * niveliInteligjences
- Efikasiteti: punaDitore / 2
  (sa më i zgjuar roboti, aq më i vlefshëm)

3.3. RoboAnalize – trashëgon RoboAdministrativ

Përdoret për analiza teknike dhe statistike.

Atribut:

- int nrAnalizave

Rregulla:

- Puna ditore: nrAnalizave * 2 + niveliInteligjences
- Efikasiteti: punaDitore + niveliInteligjences

4. Klasa Fabrika

Kjo klasë përmban robotët e fabrikës.

Atribute:

- String emri
- Robo[] robotet
- int index (pozicioni aktual për shtim)

Metodat:

4.1. Konstruktor

Pranon emrin dhe kapacitetin maksimal të robotëve.

4.2. boolean shtoRobon(Robo r)

- Shton robotin në array nëse:
    - nuk ka arritur kapacitetin,
    - nuk ekziston robot me të njëjtin id.

4.3. Robo gjejMeKonsumMinimal()

- Kthen robotin aktiv që ka konsumin më të ulët.

4.4. void shtypVetemAktivet()

- Shtyp të gjithë robotët që janë aktivë sipas rregullave të llojit të tyre.

5. Përzgjedhja optimale e robotëve nën buxhet energjetik

Duhet të implementohet një metodë:

Robo[] zgjedhRobotetOptimal(double buxheti)

Kjo metodë duhet të:

- marrë si parametër buxhetin total të energjisë,
- përzgjedhë një grup robotësh që:
    - nuk e tejkalojnë këtë buxhet,
    - maksimizojnë efikasitetin e përgjithshëm,
    - përfshijnë vetëm robotë që janë aktivë dhe mund të punojnë.

Metoda duhet të implementohet duke përdorur:

- vetëm array,
- cikle,

- krahasime,
- logjikë të krijuar brenda detyrës, pa struktura të avancuara.

Duhet të ndërtohët një algoritëm funksional bazuar në logjikë, duke analizuar konsumin, efikasitetin dhe kufizimet e buxhetit energjetik.

6. Metoda main() – Testimi i detyrës

Në metodën main() duhet të kryhen këto hapa:

1. Të krijohet një objekt i klasës Fabrika me p.sh. kapacitet 12 robotë.
2. Të shtohen të paktën:
    - 2 robotë sigurie,
    - 2 robotë prodhimi,
    - 2 robotë asistence,
    - 2 robotë administrativë (RoboMenaxhimi dhe RoboAnalize).
3. Të thirret metoda:
    - shtypVetemAktivet()
    - gjejMeKonsumMinimal()
4. Të përcaktohet një buxhet ditor (p.sh. 30 kWh).
5. Të thirret zgjedhRobotetOptimal(buxheti) dhe të shtypet:
    - lista e robotëve të zgjedhur,
    - konsumi total,
    - efikasiteti total.

Rregulla të përgjithshme:

- *Detyra duhet të jetë e organizuar në disa klasa sipas kërkesave.*
- *Duhet të përdoret trashëgimia korrekte midis klasave:*
    - *Robo → RoboSigurie*
    - *Robo → RoboProdhimi*
    - *Robo → RoboAsistence*
    - *Robo → RoboAdministrativ → RoboMenaxhimi*
    - *Robo → RoboAdministrativ → RoboAnalize*
- *Nuk lejohet përdorimi i strukturave të avancuara (`List`, `ArrayList`, `Map`, `Stream`).*
- *Duhet të përdoren vetëm `array`, `for`, `if` dhe logjikë e vetë-ndërtuar.*
- *Kodet duhet të jenë të komentuara qartë.*